ARM PrimeCell Driver

TIMER v2.0 (PL002)

API Specification

PrimeCell driver API description

Proprietary notice

Document confidentiality status

This document is Open Access. This document has no restriction on distribution.

Product status

The information in this document is Final (information on a developed product).

Web address

http://www.arm.com

Feedback

ARM limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Document Summary

Document Issue	A
PrimeCell driver identifier	TIMER
Code Version	2.0
PrimeCell identifier(s)	PL002

Contents

1 ABOUT THIS DOCUMENT

1.1 Change control

1.1.1 Change history

Issue	Date	Change
A	12/10/01	Release of TIMER Driver to support ADK timers

2 SUMMARY

This TIMER driver supports two forms of timer peripheral:

The Integrator AP timer incorporated into the system controller FPGA which contains three 16-bit counter timers. The system bus clock (which is assumed to be 20MHz) clocks Timer 0 and a fixed frequency of 24MHz is used to clock Timer 1 and Timer 2. The counters can be clocked directly or with a divide by 16 or 256 clock. The timers provide two operating modes free running and periodic. If the timer is operating in free running mode, it wraps round and continues to decrement from the maximum register value. If the timer is operating in periodic mode, it reloads the value held in the load register and continues to decrement. In this mode the timer is able to generate a periodic interrupt.

The ADK timer is a dual timer APB slave module that provides access to two interrupt generating programmable 32 bit free running counters. The system clock (PCLK) is used to control the programmable registers, and a second clock is used to drive the counter, allowing the counters to run from a much slower clock than the system clock. The timer provides three modes of operation free running, periodic and one shot. If the timer is operating in free running mode, the counter wraps after reaching its zero value, and continues to count down from the maximum value. In periodic mode the counter generates an interrupt at a constant interval, reloading the original value after wrapping past zero. In one shot mode the counter generates an interrupt once. When the counter reaches zero, it halts until re-programmed by the user.

3 FEATURES

The main features supported by the PrimeCell TIMER driver are:

- Support for Integrator AP timer
- Support for ADK timer
- Provides 4 timer modes
- Periodic timer
- One shot timer mode
- Wrap around timer mode
- Square wave timer mode

4 DRIVER USAGE

4.1 Driver Initialisation

Before a timer can be used it must be initialised.

The PrimeCell driver software by default initialises Timer 0 in test.c and assumes an integrator platform timer. Additional timers must be initialised before use by calling the following function.

- Call apTIMER _Initialize(), specifying the peripheral base address and interrupts.
  The following initialisation data may be specified using a structure passed by pointer in the final parameter:
  - Clock Clock frequency applied to timer counters.
  - eReserve If timer should be reserved on initialization.
  - eSize Size of counters, 16 bit for integrator and 16 or 32 bit for ADK timers.

4.2 Timer Operation

4.2.1 Timer Enable

To enable a timer the function apTIMER_IntervalSet is called, this disables the timer if running and sets the timer to periodic mode. It then calculates the required number of clock ticks from the interval and units specified and sets the appropriate timer pre-scale and load values to achieve this interval as accurately as possible.

Note that for the integrator AP where the timer is 16 bits and a typical value for the external timer clock is 24 MHz, then the maximum basic interval possible, with divide-by-256 pre-scale, is 0.7 seconds. However, a secondary software count has been implemented to allow extended timing of many days. However, for times of longer than a minute it is recommended that the alarms from the Real Time Clock peripheral be used instead of a timer. These will use less power, and will not be affected by the user powering down the system in the meantime.

The following parameters are passed to the function:

rHandler – pointer to the call back function

HandlerParam – This is a user supplied parameter which is normally the identifier of the peripheral using the timer.

pIntervalData – This is a pointer to the structure which holds the set-up data for the timer function required.

The interval data required to set up the timer is specified in a structure of type sIntervalData and contains the following information

- eMode - Timer mode required.
- Interval - Interval required in the specified units.
- eIntervalUnits - Units of required interval.
- LoInterval - Second interval for square wave mode.
- eLoIntervalUnits - Units of second interval for square wave mode.
- RepeatCount - Count of number of times interval is to be repeated.
- eStart - Command to start timer immediately after set up.

An alternative method of specifying a timer interval is provided by apTIMER_FrequencySet, which allows a frequency in Hz to be specified. This only provides the periodic timer mode and will run forever or until

apTIMER_Stop is called. The function apTIMER_FrequencyGet will return the timer interval specified by apTIMER_IntervalSet or apTIMER_FrequencySet as a frequency in Hz.

4.2.2 Timer Disable

To stop a timer call function apTIMER_Stop.

4.3 Timer Modes

4.3.1 Periodic Mode

This is the normal mode of operation were the counter generates an interrupt at a constant interval, reloading the original value after wrapping past zero. If a repeat count is specified other than apTIMER_REPEAT_FOREVER then the interrupt is generated for the specified number of times.

4.3.2 One Shot Mode

In this mode the counter generates an interrupt once. When the counter reaches zero, it halts until reprogrammed by the user.

4.3.3 Wrap Around Mode

In this mode the counter generates an interrupt at a constant interval for a specified number of repeats and then loads the maximum value and continues generating interrupts forever reloading the maximum value each time. This can cause problems if the timer is running in 16-bit mode with a high clock (24MHz) rate the interval after the specified number of repeats can be very short. For the ADK timers in 32-bit mode and if a low clock rate is used (32KHz) a very long timer interval can be loaded.

4.3.4 Square Wave Mode

In this mode an asymmetric timing waveform can be generated of two different timing periods, one specified by setting the standard timer interval and the other by specifying the low interval in the structure.

Note: The waveform commences with an interval time set by the LoInterval specified in the structure passed to apTIMER_IntervalSet.

4.4 Timer Allocation

To aid timer allocation from a pool of timers a timer can be reserved upon initialisation by setting the eReserve flag when calling apTIMER_Initialize. The timer can be reserved after initialization by using apTIMER_SetReserve and freed when not allocated with apTIMER_SetFree. The state of the timer can be checked at any time by using apTIMER_CheckStatus, which will return apERR_BUSY if the timer is reserved or enabled.

If a timer is enabled it must be disabled by calling apTIMER_Stop before apTIMER_SetReserve or apTIMER_SetFree can be applied successfully.

4.5 Low Level Driver Functions

To access a timer without relying on the calculations performed in apTIMER_IntervalSet a series of low level functions are provided. The timer must be initialised using apTIMER_Initialize and cannot provided the extended count facilities for a timer with only a 16-bit counter. apTIMER_LoadBackground is only supported by the ADK timer. Repeat functionality is not supported since repeat is set to 1 or forever depending on mode selected.

- apTIMER_Load - Loads a number of clock ticks into the load register.
- apTIMER_LoadBackground - Loads a number of clock ticks into the background load register.
- apTIMER_PrescaleSet - Sets the required pre-scale value in the control register.
- apTIMER_PrescaleGet - Gets the pre-scale value from the control register.
- apTIMER_CallbackSet - Sets the pointer to the interrupt call back function.
- apTIMER_ModeSet - Sets the required mode. Square wave mode not supported.
- apTIMER_Start - Starts timer.
- apTIMER_Stop - Stops timer.
- apTIMER_Clear - Clears interrupt register.
- apTIMER_ValueGet - Reads current value from timer.

4.6 ADK Timer configuration

- To enable the driver for ADK timer support the standard file applatfm.f must include the following statement:

```
#define apTIMER_VERSION 04  //This is for the ADK timers
```

- The two ADK timers can be instantiated in a system with a combined interrupt this would be specified in applatfm.h as follows:

```
apOS_VIC_TIMER0 = apOS_INTCTL_VIC | 0x04, // ADK TIMER 0 interrupt source
apOS_VIC_TIMER1 = apOS_INTCTL_VIC | 0x04, // ADK TIMER 1 interrupt source
```

When an interrupt occurs the driver interrupt handler receives the same peripheral id irrespective of which timer interrupted. The interrupt id will depend on the order of initialisation with the last id being the one associated with the interrupt.

So that the interrupt handler can determine which timer interrupted, the ids must be specified in pairs i.e. (0,1) (2,3) (4,5) etc. The handler will then check both odd and even ids.

If NO_STATIC_STATE is TRUE then there is no reliable way to determine the correct memory address for the state structure of one of the timers so separate interrupt lines must be specified.

- Since two timers are contained in one hardware block then the hardware addresses must be specified as follows:

```
apOS_SYSTEM_BASE_TIMER0 = (WORD32) 0xC0600000, // ADK Timer 0
apOS_SYSTEM_BASE_TIMER1 = (WORD32)(apOS_SYSTEM_BASE_TIMER0 + 0x20) // ADK Timer 1
```

- There will only be one peripheral id block for two timers, therefore if the timers are called from TEST_MODULE then only timer 1 can specify a peripheral id check as follows:

```
#define TEST_ALL_MODULES \
TEST_MODULE(TIMER, 804, apOS_INSTANCE(TIMER,0), apOS_SYSTEM_BASE_TIMER0, apOS_VIC_TIMER0);\
TEST_MODULE(TIMER,   0, apOS_INSTANCE(TIMER,1), apOS_SYSTEM_BASE_TIMER1, apOS_VIC_TIMER1);
```

5 DETAILS

- Peak performance obtained in tests: N/A
- ROM size (RO code + RO data) (release build): 2888 Bytes
- RAM size (ZI data) per instance (release build): 56 Bytes
- Does this driver support multiple PrimeCells?: YES
- Can these be detected at run-time? NO
- Does this driver support both endian-nesses?[1] YES
- Does this driver support Thumb?[2] YES

6 RELEASED FILES

PL002 –ISPC-1000	This API document
PL002 –TREP-1000	Test Results document
apTIMER /apTIMER .h	Public header file
apTIMER /TIMER .h	Private header file
apTIMER /TIMER .c	Source code
apTIMER /TIMER tst.c	Self-test module

1 To support both endian-nesses, the driver must be designed to be re-compiled in little-endian or big-endian form.

2 The driver may include ARM code, but this must interwork with a Thumb build.

7 PUBLIC API

7.1 Type Definitions

7.1.1 apTIMER_eMode

This defines the possible timer running modes which basically determine where the value which is loaded into the timer when it wraps around (reaches zero) comes from. Periodic is one of the hardware modes, the other modes are created by the driver.

Declaration

```
enum apTIMER_eMode
```

Elements

apTIMER_MODE_SINGLE_SHOT	The timer generates an interrupt once.
apTIMER_MODE_PERIODIC	The timer generates an interrupt at a constant interval, reloading the original value.
apTIMER_MODE_WRAPAROUND	The timer generates an interrupt at a constant interval, reloading the original value for a given number of repeat cycles it then loads the maximum count value and free runs forever.
apTIMER_MODE_SQUARE_WAVE	The timer generates interrupts at an alternating low or high interval for a given number of repeat cycles

7.1.2 apTIMER_ePrescale

The Prescale defines the possible value the input clock to the timer block is divided by before clocking the actual timer counter.

```
enum apTIMER_ePrescale
```

Elements

apTIMER_DIVIDE_BY_1	divide by 1
apTIMER_DIVIDE_BY_16	divide by 16
apTIMER_DIVIDE_BY_256	divide by 256

7.1.3 apTIMER_eSize

Used to set the size of the timer.

```
enum apTIMER_eSize
```

Elements

apTIMER_16_BIT	Default size 16 bit
apTIMER_32_BIT	For use with 32bit ADK timer

7.1.4 apTIMER_eStart

Used to set whether the timer is started when the interval is set or whether the interval is just set and the timer not started'

```
enum apTIMER_eStart
```

Elements

apTIMER_SET_AND_START	Timer set and started
apTIMER_SET_AND_NO_START	Timer set and not started

7.1.5 apTIMER_eReserve

Used to set the timer being initialized to reserved so that no other application should attempt to use it.

```
enum apTIMER_eReserve
```

Elements

apTIMER_RESERVE_ON_INIT	Timer reserved on initialization
apTIMER_NO_RESERVE_ON_INIT	Timer not reserved

7.1.6 apTIMER_eTimerUnits

Used by apTIMER_IntervalSet to specify timer interval in alternative units to default of Seconds.

```
enum apTIMER_eTimeUnits
```

Elements

apTIMER_NANOSECONDS	Time specified in nanoseconds
apTIMER_MICROSECONDS	Time specified in microseconds
apTIMER_MILLISECONDS	Time specified in milliseconds
apTIMER_SECONDS	Time specified in seconds

7.1.7 apTIMER_eCallbackCondition

This specifies the condition code returned from the interrupt handler and passed to the interrupt Handler callback function.

Declaration

```
enum apTIMER_eCallbackCondition
```

Elements

apTIMER_CONTINUE	The timer will continue with next period.
apTIMER_END	The timer will stop.
apTIMER_LOW_END	The timer has completed the low period in square wave mode
apTIMER_HIGH_END	The timer has completed the high period in square wave mode

7.1.8 apTIMER_sInitialData

This is the initial data block that should be passed in when the timer is initialized.

Declaration

```
struct apTIMER_sInitialData
{
    UWORD32               Clock;
    apTIMER_eReserve      eReserve;
    apTIMER_eSize         eSize;
}
```

Elements

Clock	Reference clock frequency applied to the Timer.
eReserve	Sets the reserve flag so that another user cannot initialize it.
eSize	Sets size of ADK timer to 16 or 32 bit counter.

Implementation

These are device parameters that are either defined by the system hardware or not expected to change, except through rebuilding the system.

7.1.9 apTIMER_sIntervalData

The interval data block is passed into the interval set function to describe the type of interval that is required.

Declaration

```
struct apTIMER_sIntervalData
{
    apTIMER_eMode       eMode;
    UWORD32             Interval;
    apTIMER_eUnits      eIntervalUnits;
    UWORD32             LoInterval;
    apTIMER_eUnits      eLoIntervalUnits;
    UWORD32             RepeatCount;
    apTIMER_eStart      eStart;
}
```

Elements

eMode	This defines the timing mode required.
Interval	This defines the normal timing interval for the timer.
eIntervalUnits	This defines the units in which the interval is defined.
LoInterval	In Square wave mode this defines the low timing interval.
eLoIntervalUnits	In Square wave mode this defines the units in which the Low interval is defined
RepeatCount	This sets the number of time the interval is to be repeated
eStart	Command to specify if interval set and start immediately.

7.1.10 apTIMER_rCallBackHandler

This is the type for the Timer callback routine.

Declaration

```
typedef void (*apTIMER_rCallBackHandler) (UWORD32 Param,
                                          apTIMER_eCallbackCondition eCondition);
```

Parameters

Param	User specified parameter to be passed to callback routine. Normally used for ID of peripheral using timer.
eCondition	Condition code returned from interrupt handler.

Implementation

Used by various routines to define a user handler to be called every time the timer interrupt occurs.

7.2 MACROS

7.2.1 apTIMER_AP

Used to define timer variant as Integrator AP timer.

Declaration

```
#define apTIMER_AP  0x00
```

7.2.2 apTIMER_ADK

Used to define timer variant as ADK timer. This value must be entered in applatfm.h if apTIMER_VERSION is defined and ADK timers are required.

Declaration

```
#define apTIMER_ADK 0x04
```

7.2.3 apTIMER_REPEAT_FOREVER

If used as a repeat value as a parameter to apTIMER_IntervalSet, this indicates that the number of times to repeat the operation is infinite.

Declaration

```
#define apTIMER_REPEAT_FOREVER 0xFFFFFFFF
```

7.3 External Functions

7.3.1 apTIMER_CallbackSet

Registers the user defined handler function as the function to be called when a pre-programmed timer event occurs.

Declaration

```
PUBLIC void apTIMER_CallbackSet (apOS_TIMER_oId oId,
                                 apTIMER_rCallBackHandler rHandler);
```

Parameters

oId	Timer identifier.
HandlerParam	Parameter to be passed to the user callback routine
rHandler	User defined routine to be called by interrupt handler; setting it to aNULL will clear the callback.

Implementation

Stores the handler function pointer for use by the interrupt handler. This action overwrites any previously registered function pointer.

Note: The registered user handler is not necessarily called every time a processor timer interrupt occurs. In the case where the programmed interval is longer than the maximum timer interval, multiple interrupts may occur before the programmed event occurs.

7.3.2 apTIMER_CheckStatus

Function to check the status of a timer. Returns apERR_NONE if free and apERR_BUSY if it is reserved or currently enabled and therefore running.

Declaration

```
PUBLIC apError apTIMER_CheckStatus (apOS_TIMER_oId oId);
```

Parameters

oId	Timer identifier.

Return Value

apERR_NONE if free and not enabled.

apERR_BUSY if reserved or enabled.

7.3.3 apTIMER_Clear

Clears the interrupt for the specified timer.

Declaration

```
PUBLIC void apTIMER_Clear (apOS_TIMER_oId oId);
```

Parameters

oId	Timer identifier.

Implementation

Writes a value to the "Clear" hardware register.

7.3.4 apTIMER_FrequencyGet

Gets the frequency of the specified timer.

Declaration

```
PUBLIC double apTIMER_FrequencyGet (apOS_TIMER_oId oId);
```

Parameters

oId	Timer identifier.

Return Value

The actual frequency set for the timer

Implementation

This is the compliment to apTIMER_FrequencySet, the actual set frequency (to the timer's best resolution) is returned

7.3.5 apTIMER_FrequencySet

Sets the frequency of the specified timer to call the registered user event handler <Frequency> times a second. Will continue until the timer is disabled, or a new interval or frequency is set. This call will supercede any previous intervals or frequencies set for this particular timer.

Declaration

```
PUBLIC void apTIMER_FrequencySet   (apOS_TIMER_oId oId,
                                    apTIMER_rCallBackHandler rHandler,
                                    UWORD32 HandlerParam,
                                    double Frequency);
```

Parameters

oId	Timer identifier.
rHandler	User defined routine to be called by interrupt handler.
HandlerParam	Parameter to be passed to the user callback routine.
Frequency	Timer frequency in Hz. Must be greater than zero.

7.3.6 apTIMER_Initialize

Initializes the timer specified. Call this function once for each timer in the system before using any of the timer driver functions.

Declaration

```
PUBLIC void apTIMER_Initialize (apOS_TIMER_oId oId,
                                apOS_System_eBaseAddress eBase,
                                UWORD32 Interrupts,
                                CONST apOS_INT_oInterruptSource * pSources,
                                apTIMER_sInitialData *pInitial);
```

Parameters

oId	Timer identifier.
eBase	Base address of timer registers.
Interrupts	Number of interrupt events recognized by handler - pass in 1.
pSources	Pointer to interrupt event.
pInitial	Pointer to block of initial data containing Clock Frequency and whether or not to reserve timer

Implementation

This ensures that the timer specified is initialized, and that all system specific information required by the driver has been passed. It also registers the interrupt handler for this timer with the interrupt module.

7.3.7 apTIMER_IntHandler

This is the standard interrupt handler for the module

Declaration

```
PUBLIC void apTIMER_IntHandler(CONST apOS_INT_oInterruptSource oInterruptId, UWORD32
DeviceId);
```

Parameters

oInterruptId	The ID of the interrupt.
DeviceId	Identifier for the instance of the driver.

Note: *NOT FOR GENERAL USE. This routine should only be called by an interrupt dispatcher*

7.3.8 apTIMER_IntervalSet

Sets the timer to produce a timed event defined in the structure sIntervalData. This call will supercede any previous intervals or frequencies set for this particular timer.

Declaration

```
PUBLIC void apTIMER_IntervalSet (apOS_TIMER_oId  oId,
                                 apTIMER_rCallBackHandler rHandler,
                                 UWORD32 HandlerParam,
                                 apTIMER_sIntervalData *pIntervalData);
```

Parameters

oId	Timer identifier.
rHandler	User defined routine to be called by interrupt handler.
HandlerParam	Parameter to be passed to the user callback routine.
pIntervalData	Pointer to the structure containing the information about the interval to set.

Implementation

Disables the timer.

Sets the timer to the periodic mode.

Calculates the number of clock ticks required from the interval specified, and sets appropriate timer pre-scale and load values to achieve this interval as accurately as possible. Note that if the timer is 16 bits and a typical value for the external timer clock is 3.6864 MHz, then the maximum basic interval possible, with divide-by-256 pre-scale, is 4.55 seconds. However, a secondary software count has been implemented to allow extended timing of many days. However, for times of longer than a minute it is recommended that the alarms from the Real Time Clock peripheral be used instead of a timer. These will use less power, and will not be affected by the user powering down the system in the meantime.

Sets the repeat count.

Stores the pointer to the call back handler.

Re-enables the timer.

When the timer generates an interrupt the interrupt handler disables the timer and calls the call back handler if specified and then re-enables the timer if required.

7.3.9 apTIMER_Load

Sets a new timer counter interval.

Declaration

```
PUBLIC void apTIMER_Load (apOS_TIMER_oId oId, UWORD32 Interval);
```

Parameters

oId	Timer identifier.
Interval	New timer Interval.

Implementation

Sets the timer load hardware register, for 16-bit timers the top 16-bits must be set to zero.

7.3.10 apTIMER_LoadBackground

Available in ADK timer version only.

Loads a new interval into the background load register.

Declaration

```
PUBLIC void apTIMER_LoadBackground (apOS_TIMER_oId oId, UWORD32 Interval);
```

Parameters

oId	Timer identifier.
Interval	New timer Interval.

Implementation

Sets the timer background load hardware register. Which will be loaded into the timer when the current count has reached zero.

7.3.11 apTIMER_ModeSet

Sets the Mode of the timer.

Declaration

```
PUBLIC apError apTIMER_ModeSet (apOS_TIMER_oId  oId,
                                apTIMER_eMode eMode);
```

Parameters

oId	Timer identifier.
eMode	User defined routine to be called by interrupt handler.

Return Value

apERR_NONE if mode set.

apERR_UNSUPPORTED if mode is not supported.

Implementation

Allows the mode of the timer to be set to support the three basic modes periodic, wraparound and one-shot. The complex mode of square wave can only be set using apTIMER_IntervalSet and this function will return apERR_UNSUPPORTED if this is requested.

7.3.12 apTIMER_PrescaleGet

Reads the current timer prescale value.

Declaration

```
PUBLIC apTIMER_ePrescale apTIMER_PrescaleGet (apOS_TIMER_oId oId);
```

Parameters

oId	Timer identifier.

Return Value

Current timer prescale value
- apTIMER_DIVIDE_BY_1
- apTIMER_DIVIDE_BY_16
- apTIMER_DIVIDE_BY_256

Implementation

Reads the currently set prescale value from the timer control hardware register. The prescale value is the value by which the clock into the timer hardware block is divided down by before being used to clock the timer counters.

7.3.13 apTIMER_PrescaleSet

Sets a new timer prescale value.

Declaration

```
PUBLIC void apTIMER_PrescaleGet (apOS_TIMER_oId oId, apTIMER_ePrescale ePrescale);
```

Parameters

oId	Timer identifier.
ePrescale	New prescale value.

Implementation

Sets the prescale value in the timer control hardware register. The prescale value is the value by which the clock into the timer hardware block is divided down by before being used to clock the timer counters.

7.3.14 apTIMER_RawISR

This is the raw interrupt handler for the module, to be called directly from the interrupt vector

Declaration

```
PUBLIC IRQ void apTIMER_RawISR(void);
```

Note: *NOT FOR GENERAL USE. This routine should only be executed as a branch from the IRQ vector*

7.3.15 apTIMER_SetFree

Function to free the timer so that it is no longer reserved.

Declaration

```
PUBLIC apError apTIMER_SetFree (apOS_TIMER_oId oId);
```

Parameters

oId	Timer identifier.

Return Value

apERR_NONE if free has been set.

apERR_BUSY if currently enabled.

7.3.16 apTIMER_SetReserve

Used to reserve the timer so that nobody else can come along and initialize it after you already have

Declaration

```
PUBLIC apError apTIMER_Reserve (apOS_TIMER_oId oId);
```

Parameters

oId	Timer identifier.

Return Value

apERR_NONE if reserve has been set.

apERR_BUSY if already reserved or currently enabled.

7.3.17 apTIMER_Start

Enables the specified timer.

Declaration

```
PUBLIC void apTIMER_Start (apOS_TIMER_oId oId);
```

Parameters

oId	Timer identifier.

Implementation

Enables the timer by writing to the timer control register.

7.3.18 apTIMER_StateSizeGet

Finds the amount of space required for driver state data

Declaration

```
PUBLIC UWORD32 apTIMER_StateSizeGet(void);
```

Return Value

Size (in bytes) required.

Implementation

This function is required if apOS_NO_STATIC_STATE is defined as TRUE as it will retrieve the size required for storage of the driver state data

7.3.19 apTIMER_Stop

Disables the specified timer.

Declaration

```
PUBLIC void apTIMER_Stop (apOS_TIMER_oId oId);
```

Parameters

oId	Timer identifier.

Implementation

Disables the timer by writing to the timer control register.

7.3.20 apTIMER_ValueGet

Reads the current timer counter value.

Declaration

```
PUBLIC UWORD32 apTIMER_ValueGet (apOS_TIMER_oId oId);
```

Parameters

oId — Timer identifier.

Return Value

Current timer value.

Implementation

Reads the timer value hardware register, and returns the 32-bit value (for 16-bit timers the top 16-bits are returned as zero).